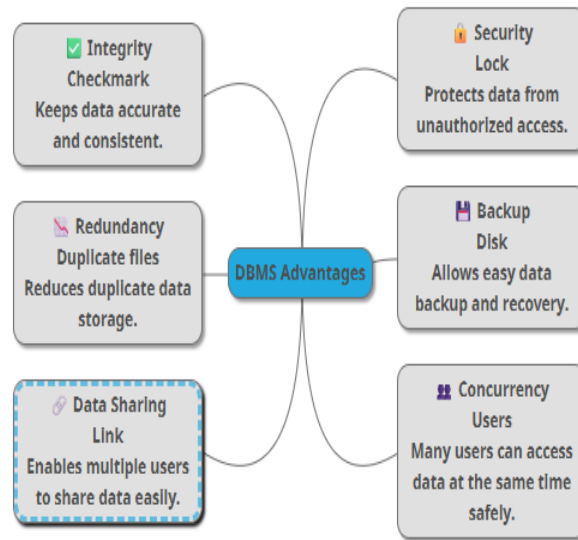


Database

1. Flat File Systems vs. Relational Databases

	Flat File Systems	Databases
Structure	Data is stored in a single table or file as plain text. Each line represents a record, and fields are separated by a delimiter (e.g., comma, tab). Simple structure, no complex indexing.(All data in one file, like a list)	Data is organized into multiple related tables with rows and columns. A schema defines tables, columns, and data types. Managed by an RDBMS (Data stored in many related tables.)
Data Redundancy	Same data repeated many times (High redundancy — the same information (like customer address) may appear in many records, leading to inconsistencies and wasted space)	Redundancy minimized through normalization — data stored once and referenced using keys. Ensures consistency and saves storage. Redundancy minimized through normalization — data stored once and referenced using keys. Ensures consistency and saves storage.(Data stored once and shared.)
Relationships	No built-in way to define or enforce relationships between records. Any links must be handled by the application.	Relationships are defined using primary and foreign keys , allowing one-to-one, one-to-many, and many-to-many links. Enforced by the RDBMS.
Example Usage	• Contact lists • Configuration files (.ini, .json) • CSV/TSV data exchange • Log files	• Enterprise applications (CRM, ERP) • Financial and transactional systems • Business intelligence and analytics • Multi-user environments
Drawbacks	• Poor scalability • Data integrity issues • Hard to query • Weak security • Limited concurrent access	• Performance bottlenecks with complex joins • High setup/training costs • Inflexible schema • Not ideal for unstructured data Needs setup, harder to change, slower with big joins.

I. DBMS Advantages



II. ROLES IN A DATABASE SYSTEM

Role	What They Do
System Analyst	Understands the business needs and requirements. Designs how the database and applications will meet those needs. Acts as a bridge between users and developers.
Database Designer	Plans the database structure. Creates tables, relationships, keys, and schema to organize data efficiently and avoid redundancy.
Database Developer	Writes code to build and implement the database. Creates queries, stored procedures, and scripts to manage and manipulate data.
Database Administrator (DBA)	Maintains and manages the database. Ensures security, backups, performance, and smooth operation. Handles user access and troubleshooting.
BI (Business Intelligence) Developer	Analyzes data in the database to create reports, dashboards, and insights. Helps businesses make decisions using data trends and visualization.
Application Developer	Builds software applications that use the database. Connects the database to user interfaces and ensures data can be read, added, updated, or deleted correctly.

III. RELATIONAL VS. NON-RELATIONAL DATABASES

Type	Description	Examples	Use Case Examples
------	-------------	----------	-------------------

Relational (RDBMS)	Data is stored in tables with rows and columns. Uses structured schema and SQL. Supports relationships with keys.	MySQL, PostgreSQL, Oracle	Banking systems, e-commerce platforms, inventory management
Non-Relational (NoSQL)	Data is stored in flexible formats (documents, key-value, graph, column). Schema-less or dynamic schema.	MongoDB, Cassandra, Redis	Social media apps, real-time analytics, IoT data, large-scale data streaming

IV. CENTRALIZED VS. DISTRIBUTED VS. CLOUD DATABASES

Type	Description	Use Case Examples
Centralized	Database exists on a single server or location. All users connect to that one server.	Small business inventory, internal company apps
Distributed	Database spread across multiple servers or locations. Data is replicated or partitioned for performance and reliability.	Global e-commerce, multinational banking, large social networks
Cloud	Database hosted on cloud provider (AWS, Azure, Google Cloud). Accessible via internet. Can be centralized or distributed in the cloud.	SaaS apps, online collaboration tools, mobile apps

V. 5. CLOUD STORAGE AND DATABASES

A. What is Cloud Storage?

Cloud storage is online storage managed by cloud providers where data is stored on remote servers instead of local hardware.

It supports databases by providing scalable storage, high availability, and backup/recovery options.

B. Advantages of Cloud-Based Databases

- Scalability: Easily increase storage or compute as needed.
- Accessibility: Access from anywhere via the internet.
- Maintenance: Provider handles updates, patches, and hardware.
- High Availability: Built-in replication and failover reduce downtime.
- Cost-Efficient: Pay-as-you-go pricing (no need to buy expensive servers).

Examples: Azure SQL Database, Amazon RDS, Google Cloud Spanner

C. Disadvantages / Challenges

- Internet dependency: Cannot access database if internet is down.
- Security concerns: Data is off-premises, so strong encryption and access control needed.
- Limited control: Less control over underlying hardware and configuration.

- Potential latency: For very large or sensitive applications, cloud access might be slower than local servers.